ERHVERVSAKADEMI KØBENHAVN

Tutor-Opgaver [Uge 9]

Opgave 1: Match-Case/if-statment Menu

Opret en variabel user_role. Brug match-case syntaksen til at printe forskellige velkomstbeskeder afhængigt af, om rollen er "admin", "user" eller noget tredje (default)

Opgave 2: Loop Around (For-loop & Range)

Brug et for-loop og range() til at tælle fra 0 til 50. Hvis tallet er 10, skal du bruge continue til at springe det over og printe "we don't print '10'". Hvis tallet når 45, skal du bruge break til at stoppe loopet

Opgave 3: Det uendelige loop (While)

Lav et uendeligt while True-loop, der beder brugeren om at indtaste en kommando. Programmet skal printe "You typed: [kommando]". Hvis brugeren skriver "quit", skal programmet afsluttes (brug break eller exit()).

Opgave 4: Din første funktion (Return)

Definér en funktion calculate_area, der tager en parameter radius.

Funktionen skal beregne og returnere arealet af en cirkel (Brug variablen ***PI = 3.14159***). Kald funktionen for en radius på 5, og print resultatet.

Opgave 5: Sikker Division (Fejlhåndtering med Try-Except)

Byg et program, der beder brugeren indtaste et tal, som 10 skal divideres med. Brug try-except til at sikre, at programmet ikke crasher, hvis brugeren indtaster tekst (ValueError) eller nul (ZeroDivisionError).

Opgave: The Secure Calculator & Logger[pt. 0]

Scenario:

Du skal bygge en robust lommeregner, der ikke crasher, uanset hvad brugeren taster. Programmet skal også føre en log over alle succesfulde og fejlslagne beregninger for at demonstrere, hvordan man manipulerer objekter via funktioner.

(Obs. Husk at tjekke hele opgaven igennem inden du/I begynder på den)

Opgave: The Secure Calculator & Logger[pt. 1]

1. Opret en global liste kaldet ***operation_log = []***

2. Opret en funktion ***log_action(log_list, action)***. Denne funktion skal bruge .append() til at tilføje strengen action til log_list. (Husk: Da lister er "mutable" objekter, vil ændringen i funktionen også påvirke den globale liste).

3. Opret en funktion ***get_valid_number(prompt_text)***.

Opgave: The Secure Calculator & Logger[pt. 2]

4. Funktionen skal bruge et uendeligt while True-loop til at bede om et tal ved at bruge prompt_text som input-tekst.

5. Hvis brugeren taster *"q"*, skal funktionen bruge exit() til at lukke hele programmet.

6. Brug try-except til at forsøge at konvertere brugerens input til en float. Hvis det mislykkes (f.eks. hvis de skriver "hej"), skal du fange ValueError og printe *"Fejl: Ugyldigt tal, prøv igen"* og lade loopet køre forfra.

Opgave: The Secure Calculator & Logger[pt. 3]

7. Hvis konverteringen lykkes, skal funktionen bruge return til at sende tallet tilbage.

8. Opret en funktion ***safe_divide(a, b)***

9. Brug en if-sætning til at tjekke, om b er lig med 0. Hvis den er, skal du bruge raise ValueError(*"Cannot divide by zero!"*) for at kaste en manuel fejl.

10. Hvis b ikke er 0, skal funktionen returnere a / b.

11. Lav en uendelig while True løkke, der viser en menu med tre valg: "1.Divider to tal", "2. Vis log", "3. Afslut".

Opgave: The Secure Calculator & Logger[pt. 4]

12. Brug match-case/if-statment til at håndtere brugerens valg.

Case 1: Brug din get_valid_number-funktion til at hente to tal (tæller og nævner). Brug en try-except blok til at kalde safe_divide. Hvis den lykkes, print resultatet og kald log_action for at logge successen. Fanges en ValueError (fra din raise i del 3), print fejlen og log den som en mislykket operation. **Case 2**: Print alle elementer i din operation_log ved hjælp af et for-loop. **Case 3:** Brug break til at stoppe løkken.